

Reflexion

Prosa:

Wenn ich mein Klassendiagramm betrachte, erkenne ich, dass mein Client im Wesentlichen so aussieht wie ursprünglich geplant. Das Paket "clientNetwork" mit all seinen Klassen hat sich zum Beispiel überhaupt nicht geändert.

Ich habe jedoch in meinem "clientModel" Paket einige Änderungen vorgenommen. Der Inhalt der Klassen ist grundsätzlich gleich geblieben, aber ich habe das Paket in zwei verschiedene Pakete aufgeteilt: "clientData" und "clientLogik". "clientData" enthält alle Klassen, die Spieldaten speichern (einschließlich des Modells des MVC - "GameDataModel" Klasse); und "clientLogik" ist für die Kartengenerierung, -validierung und Wegfindung verantwortlich (MVC). Außerdem habe ich das "Strategy Pattern" für die Wegfindung umgesetzt (wie in der Vorlesung gelernt). Ich verwende das "IChooseTarget" Interface und zwei Klassen implementieren die „chooseTarget()“-Methode, abhängig von der Spielphase. Entweder sucht die KI den Schatz und die Burg oder wechselt auf die andere Kartenhälfte.

Für die Benutzeroberfläche habe ich in meiner Implementierung die GUI ausgelassen, da es zu aufwändig war, und somit auch das "IUserInterface"-Interface. Ich hätte das Interface auch implementieren können, aber das stößt gegen das "YAGNI Prinzip" an.

Ich habe versucht, die Designprinzipien, die ich in der Vorlesung gelernt habe, einzuhalten, obwohl es nicht immer offensichtlich war, wie es richtig umzusetzen war. Ich habe möglichst viele Methoden privat gehalten, um Kapselung und Information Hiding zu gewährleisten, was auch bei der Kopplung und Kohäsion geholfen hat. Ich habe auch versucht, das "Single Responsibility Principle" zu unterstützen, indem ich alle Methoden möglichst klein geschrieben habe, um immer genau eine Aufgabe zu erledigen.

Y-Statements:

1. Im Kontext des UI-Designs habe ich mich aufgrund begrenzter Zeit dafür entschieden, GUI zu vernachlässigen und das „IUserInterface“-Interface nicht zu implementieren, um die Umsetzung des Systems zu erleichtern und den Aufwand zu reduzieren. Das hat den Nachteil, dass die Benutzerfreundlichkeit des Systems möglicherweise beeinträchtigt wird.

2. Bei der Implementierung der Wegfindung habe ich mich aufgrund von Anforderungen an die Flexibilität und Erweiterbarkeit des Systems für das „Strategy Pattern“ entschieden. Dies ermöglicht es, verschiedene Strategien zur Wegfindung zu implementieren und leicht auszutauschen. Dabei habe ich jedoch in Kauf genommen, dass die Komplexität des Systems erhöht wird und mehr Code entsteht.

3. Im Hinblick auf die Klassen- und Methodenaufteilung habe ich mir dafür entschieden, das „clientModel“-Paket in „clientData“ und „clientLogik“ aufzuteilen, um das "Single Responsibility Principle“ zu unterstützen und die Kohäsion der Klassen zu erhöhen. Dadurch können Änderungen an einer Klasse leichter vorgenommen werden, ohne dass dies Auswirkungen auf andere Klassen hat. Allerdings habe ich dabei die Kopplung der Klassen in Kauf genommen, da einige Klassen nun voneinander abhängig sind.